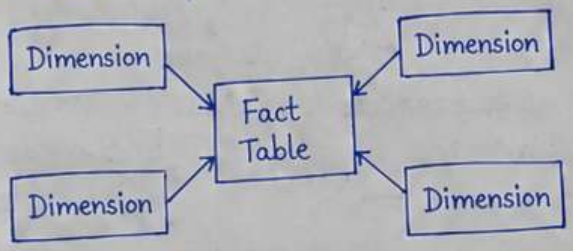
## 1) Differentiate between star and snowflake schemas. Why is a star schema preferred in Power BI?

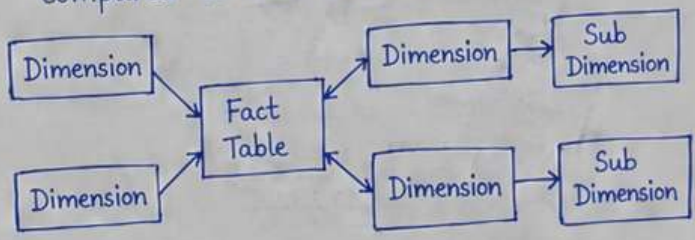
### Star Schema

- One central fact table connected directly to dimension tables.
- Simple structure.
- Fewer joins.
- Better performance.
- Less complex.




### Snowflake Schema

- Dimension tables are further divided into related tables.
- More complex structure.
- More joins.
- Less data redundancy.
- Performance is relatively lower compared to star schema.




### Why star schema is preferred in Power BI?

- Simplifies data modelling.
- Improves query performance.
- Makes DAX calculations easier.
- Easier to understand and maintain.

---

## 2) What is a role-playing dimension? How is USERELATIONSHIP used with it?

- A role-playing dimension is a single dimension table used for multiple purposes.
- Eg: A Date table can represent Order Date, Ship Date and Delivery Date.
- In data model, only one relationship can be active at a time.
- USERELATIONSHIP() activates an inactive relationship temporarily within a calculation.

**Example:**

```
Sales by Ship Date =
    CALCULATE(
        [Total Sales],
        USERELATIONSHIP(
            Sales[ShipDate], 'Date'[Date]
        )
    )
```

This measure calculates sales according to Ship Date instead of the active Order Date relationship.

---

## 3) Why is a date table required for time intelligence? How is it created and marked?

- A Date table contains continuous dates and columns such as Year, Month and Quarter.
- It is required for time-intelligence calculations like YTD, previous year and YoY growth.
- Create using DAX: `Date = CALENDARAUTO()`
- After creation, add required date columns (Year, Month, Quarter etc).
- Ensure dates are unique and continuous.
- Mark as date table: Table Tools → Mark as date table → Select the Date column.

4) Write DAX measures for Sales YTD, Sales Last Year, and Year-on-Year Growth percentage.

These measures are used to analyse sales performance over time.

- Total Sales :

```
Total Sales =
  SUM(Sales[SalesAmount])
```

- Sales YTD :

```
Sales YTD =
  TOTALYTD(
    [Total Sales],
    'Date'[Date]
  )
```

- Sales Last Year :

```
Sales Last Year =
 CALCULATE(
    [Total Sales],
    SAMEPERIODLASTYEAR(
      'Date'[Date]
    )
 )
```

- YoY Growth % :

```
YoY Growth % =
  DIVIDE(
    [Total Sales] - [Sales Last Year],
    [Sales Last Year],
    0
  )
```

↳ Format this measure as Percentage.

Note : These calculations require a properly configured Date table.

---

5) Explain filter context, row context, and context transition with examples.

In DAX, context determines which data is used in a calculation.

- Filter Context : Filters applied through slicers, visuals or DAX. It filters the data before the calculation is performed.
  Example : If a slicer selects City = Lucknow, the measure will calculate sales only for Lucknow.
- Row Context : Evaluates an expression for each row in a table.
  Example : In a calculated column, Quantity x Price is calculated for every row.
- Context Transition : Converts row context into filter context, usually through CALCULATE(). It allows a row context to be used as a filter context.

Example :

```
Total Sales =
  SUM(Sales[Amount])
```

↳ When used in a city-wise visual, the measure calculates sales for each city according to its filter context.

---

6) Explain CALCULATE with at least two examples, and differentiate between ALL and ALLSELECTED.

CALCULATE() evaluates an expression after modifying its filter context.

- Example 1 - Sales for 2026 :

```
Sales 2026 =
  CALCULATE(
    [Total Sales],
    'Date'[Year] = 2026
  )
```

- Example 2 - Sales for a particular city :

```
Lucknow Sales =
  CALCULATE(
    [Total Sales],
    Sales[City] = "Lucknow"
  )
```

Difference :

- ALL() : Removes filters from a specified table or column.
- ALLSELECTED() : Removes relevant visual-level filters while retaining external selections, such as slicers.

ALL() is useful for grand totals, while ALLSELECTED() is useful for visual totals based on user selections.

## 7) What are iterator functions? Explain SUMX with an example.

- Iterator functions evaluate an expression for each row of a table and then combine the results.
- Examples include SUMX(), AVERAGEX() and COUNTX().
- SUMX() evaluates an expression row by row and adds all the results.

**Example :**

```
Total Revenue =
SUMX(
    Sales,
    Sales[Quantity] * Sales[UnitPrice]
)
```

Here, Power BI multiplies quantity by unit price for every row and then adds the results to calculate total revenue.

---

## 8) Describe five techniques for improving the performance of a Power BI report. How is Performance Analyzer used?

Report performance can be improved using the following techniques :

1. Use a star schema for simple data modelling.
2. Remove unnecessary columns and rows.
3. Prefer measures over unnecessary calculated columns.
4. Reduce the number of visuals on a page.
5. Optimise DAX formulas and data relationships.

**Performance Analyzer :**

- Performance Analyzer is a Power BI tool available under the View tab.
- It records the time taken by visuals to load and separates query time, visual display time and other processing time.
- This helps identify slow visuals and optimise them.

---

## 9) Explain slicers, drill-down, drill-through, and bookmarks in interactive dashboards.

These features make Power BI dashboards interactive.

- **Slicers** : Allow users to filter reports by fields such as date, city or product.
- **Drill-down** : Moves from summary data to detailed levels, such as Year to Quarter to Month.
- **Drill-through** : Opens another report page showing details related to a selected item.
- **Bookmarks** : Save a particular report view, including selected filters and visual states, and can be used for navigation.

These features help users explore data and understand business performance easily.

10) What is Row-Level Security ? Explain how static and dynamic RLS are created, tested, and assigned.

Row-Level Security (RLS) restricts the rows of data a user can view in a Power BI report.

- Static RLS : A fixed filter is created for a role, such as

  `Sales[City] = "Lucknow"`

- Dynamic RLS : Uses the logged-in user's identity to filter data.

  Example : `[Email] = USERPRINCIPALNAME()`

Steps to implement RLS :

1. Create roles in Power BI Desktop using Manage Roles.
2. Define filters for each role (static or dynamic).
3. Test the roles using View As.
4. Publish the report to the Power BI Service.
5. Assign users or security groups to roles in the Power BI Service.

---

11) Who is not restricted by RLS in the Power BI Service, and why ?

- RLS generally applies to users with Viewer access to a Power BI semantic model or report.
- Users with Admin, Member or Contributor roles in the workspace are not restricted by RLS when accessing the data through that workspace.
- These roles have elevated permissions to manage or edit workspace content.
- Therefore, RLS should not be treated as a security restriction for these roles.
- Users who require restricted data access should generally be assigned the Viewer role with appropriate RLS permissions.

---

12) Explain how Power Apps and Power Automate can be integrated with Power BI, with a business example.

Power Apps and Power Automate extend Power BI beyond data visualisation.

- Power Apps : Allows users to create or update records through an app embedded in a Power BI report.
- Power Automate : Automates workflows, notifications and approvals based on business events.

Example :

In a sales dashboard, a manager can use Power Apps to update a customer's status (e.g. Leads to Won). Power Automate can then send an email notification to the sales team or trigger an approval workflow based on the update.

